GLYPH

a super regular RISC architecture

Michael Clark

April 4, 2025 DRAFT

Contents

1. Architecture

1.1. Introduction

glyph is a super regular RISC architecture that encodes constants in a secondary stream accessed via an immediate base register that points at immediate blocks containing constants accessed via a constant address mode. the immediate base register branches like the program counter, and procedure call and return set and restore *(pc,ib)* together.

glyph uses relative address vectors in its link register which is different to typical RISC architectures. glyph does this so that the branch instructions can fit *(pc,ib)* into the a single link register for compatibility. glyph achieves this by packing two relative *(pc,ib)* displacements into a relative address vector.

immediate blocks can be switched using the immediate block branch instruction. immediate blocks, unlike typical RISC architectures, mean that most relocations are word sized like CISC architectures, and can use C-style structure packing and alignment rules.

this list outlines some differentiating elements of the super regular RISC architecture:

- variable length instruction format supporting 16, 32, 64, and 128-bit instructions.
- 16-bit compressed instruction packets can access 8 registers.
- *(pc,ib)* is a special program counter and immediate base register address vector.
- link register contains packed relative *(pc,ib)* address vector to function entry.
- `ibj` *immediate-block-jump* adds a relative address to the immediate base register.
- `lib` *load-immediate-block* uses an unsigned displacement to access constants.
- `jalib` *jump-and-link-immediate-block* or *call* links address vector and adds constants to *(pc,ib)* and is used to branch the program counter and immediate base register at the same time for calling procedures.
- `jtlib` *jump-to-link-immediate-block* or *ret* subtracts link vector from and adds constants to *(pc,ib)* and is used to branch the program counter and immediate base register at the same time for returning from called procedures.
- `pin` *pack-indirect* packs two absolute addresses as relative address vector from *(pc,ib)* and is used for calling absolute addresses such as virtual functions.

1.2. Instruction format

glyph has a variable length instruction format supporting 16, 32, 64, and 128-bit instruction packets. the instruction packet has been designed to use a *super regular* scheme, whereby successive instruction packets extend the fields in the previous packet.

1.2.1. instruction templates

the variable length instruction format has a single base format where fields in the template instruction form are extended by successive instruction packets.

15 7	6 2	1 0
$operand_{[8:0]}$	$opcode_{[4:0]}$	$sz_{[1:0]}$

Figure 1.1.: 16-bit instruction template.

15 7	6 2	1 0
$operand_{[8:0]}$	$opcode_{[4:0]}$	$sz_{[1:0]}$
$operand_{[17:9]}$	$opcode_{[9:5]}$	$sz_{[3:2]}$

Figure 1.2.: 32-bit instruction template.

15 7	6 2	1 0
$operand_{[8:0]}$	$opcode_{[4:0]}$	$sz_{[1:0]}$
$operand_{[17:9]}$	$opcode_{[9:5]}$	$sz_{[3:2]}$
$operand_{[26:18]}$	$opcode_{[14:10]}$	$sz_{[5:4]}$
$operand_{[35:27]}$	$opcode_{[19:15]}$	$sz_{[7:6]}$

Figure 1.3.: 64-bit instruction template.

15 7	6 2	1 0
$operand_{[8:0]}$	$opcode_{[4:0]}$	$sz_{[1:0]}$
$operand_{[17:9]}$	$opcode_{[9:5]}$	$sz_{[3:2]}$
$operand_{[26:18]}$	$opcode_{[14:10]}$	$sz_{[5:4]}$
$operand_{[35:27]}$	$opcode_{[19:15]}$	$sz_{[7:6]}$
$operand_{[44:36]}$	$opcode_{[24:20]}$	$sz_{[9:8]}$
$operand_{[53:45]}$	$opcode_{[29:25]}$	$sz_{[11:10]}$
$operand_{[62:54]}$	$opcode_{[34:30]}$	$sz_{[13:12]}$
$operand_{[71:63]}$	$opcode_{[39:35]}$	$sz_{[15:14]}$

Figure 1.4.: 128-bit instruction template.

1.2.2. size encoding

the variable length instruction format has a *2-bit* size field in a fixed position in every 16-bit instruction packet, somewhat inspired by LEB128, to reduce the complexity of variable length instruction size decoding.

Instruction Size	**Size Fields**
16-bit	{00}
32-bit	{01,11}
64-bit	{10,11,11,11}
128-bit	{11,11,11,11,11,11,11,11}

Table 1.1.: Variable-length instruction size fields

1.2.3. 16-bit instructions

the 16-bit instructions forms are super regular in that operand and opcode bits do not overlap and the number and complexity of the formats is reduced so that vectorized instruction decoding is easier in software. the scheme is designed so that *1-bit* of coding space in the larger packet can be used to extend register sizes for the 16-bit ops.

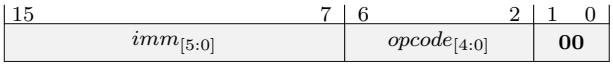


Figure 1.5.: 16-bit large immediate.

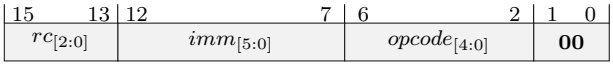


Figure 1.6.: 16-bit one operand with immediate.

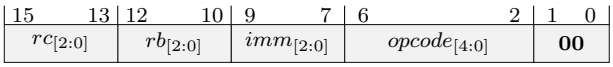


Figure 1.7.: 16-bit two operand with immediate.

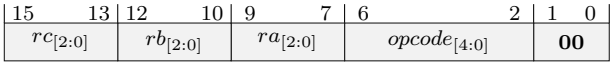


Figure 1.8.: 16-bit three operand.

1.3. Constant stream

glyph seperates the instruction stream into two streams, one with instructions and one with constants. the instruction stream is addressed with the *program counter (pc)* and the constant stream is addressed with the *immediate base register (ib)*.

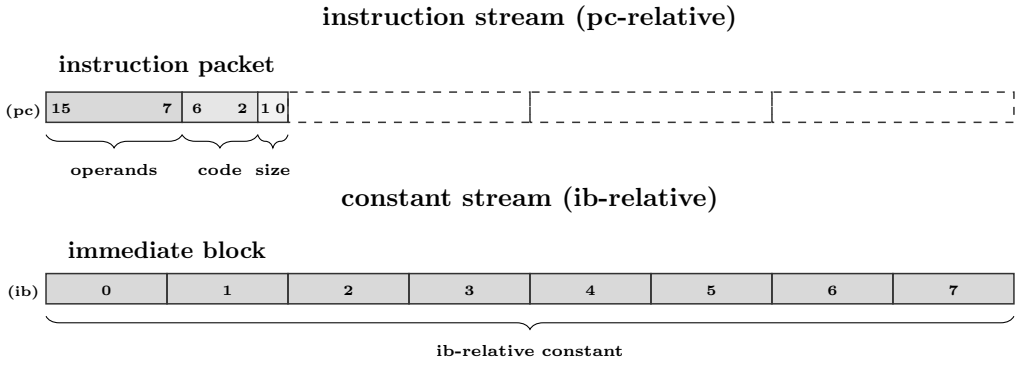


Figure 1.9.: program counter and immediate base register.

immediate blocks are aligned memory blocks addressed by the *immediate base register*.

immediate blocks can be navigated by branching the constant stream independently using the *immediate-block-jump* instruction, or together with the program counter using procedure call and return instructions that set and restore *(pc,ib)* via a link register that contains a packed relative address vector. the use of packed relative address vectors is for backward compatibility with a single link register.

for procedure calls and returns, the instruction and constant streams are set at the same time using the call and return instructions; *jump-and-link-immediate-block*, *jump-to-link-immediate-block*, which add and subtract relative address vectors to *(pc,ib)*, the program counter and the immediate base register. the *pack-indirect* instruction allows absolute *(pc,ib)* addresses to be packed into a relative address vector for indirect calls.

the instruction forms only use bonded register slots for immediate operands and operand bits do not overlap opcode bits. the use of immediate blocks means large immediate constants can all be accessed with short references encoded inside of register slots for instructions that use an immediate block relative addressing mode.

1.4. Register file

the glyph register file is extensible due to the variable length instruction format and supports a different number of registers depending on the instruction size.

- 16-bit instruction packet can access 8 registers with up to 3 operands.
- 32-bit instruction packet can access 64 registers with up to 3 operands.
- 64-bit instruction packet can access 64 registers with up to 6 operands.

the register state accessible by the 16-bit instruction packet is comprised of:

- program counter register (aligned to 2 bytes).
- immediate base register (aligned to 64 bytes).
- 8 ×general purpose registers.
- 1 ×predicate register.

the following diagram shows the register state accessible by the 16-bit instruction packet:

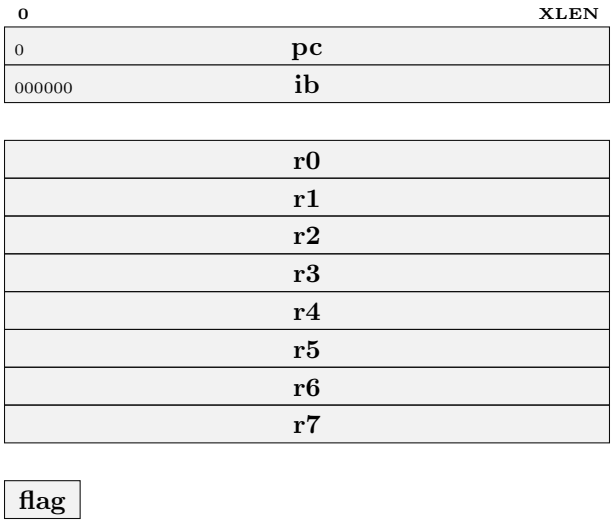


Figure 1.10.: register state accessible by 16-bit instruction packet.

2. Instructions

2.1. 16-bit opcodes

2.1.1. break

15 — 7	6 — 2	1 — 0
imm9	*opcode*	*size*
uimm	**00000**	**00**

break uimm9

The *break* instruction causes a trap to the debugger. The program counter and trap cause are saved to privileged registers for the operating system to dispatch to a debugger.

2.1.2. j

15 — 7	6 — 2	1 — 0
imm9	*opcode*	*size*
simm	**00001**	**00**

j simm9

The *j* or *jump* instruction is an unconditional branch instruction that adds a relative immediate address to the program counter. The resulting address is *pc + simm9 + 2*.

```
pc += simm9 * 2 + 2
```

2.1.3. b

15 — 7	6 — 2	1 — 0
imm9	*opcode*	*size*
simm	**00010**	**00**

b simm9

The *b* or *branch* instruction is a conditional branch instruction that adds a relative immediate address to the program counter, if the *flag* register has been set by a compare instruction. The resulting address is *pc + simm9 + 2*.

```
if flag:
    pc += simm9 * 2 + 2
```

2.1.4. ibj

15 … 7	6 … 2	1 0
imm9	*opcode*	*size*
simm	**00011**	**00**

ibj simm9

the *ibj* or *immediate-block-jump* instruction adds a 64-bit relative address to the immediate base register.

```
ib += simm9 * 64
```

2.1.5. jalib

15 … 13	12 … 7	6 … 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**uimm**	**00100**	**00**

jalib rc, ib(uimm6*8)

the *jalib* or *jump-and-link-immediate-block* instruction loads a 64-bit constant addressed by *ib + uimm6 * 8* containing a *i32x2* relative address vector, adds it to *(pc,ib)*, then saves the relative address vector to the *rc* register.

```
tmp = (i32x2)[ib + uimm6 * 8]
{rpc,rib} = (i32x2)tmp
pc += rpc + 2
ib += rib
r[rc] = tmp
```

2.1.6. jtlib

15 … 13	12 … 7	6 … 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**uimm**	**00101**	**00**

jtlib rc, ib(uimm6*8)

the *jtlib* or *jump-to-link-immediate-block* instruction loads a 64-bit constant addressed by *ib + uimm6 * 8* containing a *i32x2* relative address vector, subtracts the *i32x2* relative address vector in the *rc* register from it, then adds it to *(pc,ib)*.

```
{rpc,rib} = (i32x2)r[rc]
{dpc,dib} = (i32x2)[ib + uimm6 * 8]
pc += dpc - rpc
ib += dib - rib
```

2.1.7. lib.i64

15 13	12 7	6 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**uimm**	**00110**	**00**

lib.i64 rc, ib(uimm6*8)

the *lib* or *load-immediate-block* instruction loads a 64-bit constant addressed by *ib + uimm6 * 8* then saves it to the *rc* register.

```
r[rc] = (i64)[ib + uimm6 * 8]
```

2.1.8. li.i64

15 13	12 7	6 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**simm**	**00111**	**00**

li.i64 rc, simm6

the *li* or *load-immediate* instruction sign-extends *simm6* then saves it to the *rc* register.

```
r[rc] = simm6
```

2.1.9. addi.i64

15 13	12 7	6 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**simm**	**01000**	**00**

addi.i64 rc, simm6

the *addi* or *add-immediate* instruction sign-extends *simm6* then adds it to the *rc* register.

```
r[rc] = r[rc] + simm6
```

2.1.10. srli.i64

15 13	12 7	6 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**uimm**	**01001**	**00**

srli.i64 rc, uimm6

the *srli* or *shift-right-logical-immediate* instruction performs a logical right shift by *uimm6* bits of the value in the *rc* register. zeros are copied into the right most bits.

```
r[rc] = (u64)r[rc] >> simm6
```

2.1.11. srai.i64

15 13	12 7	6 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**uimm**	**01010**	**00**

srai.i64 rc, uimm6

the *srai* or *shift-right-arithmetic-immediate* instruction performs an arithmetic right shift by *uimm6* bits of the value in the *rc* register. sign is copied into the right most bits.

```
r[rc] = (i64)r[rc] >> simm6
```

2.1.12. slli.i64

15 13	12 7	6 2	1 0
rc	*imm6*	*opcode*	*size*
rc	**uimm**	**01011**	**00**

slli.i64 rc, uimm6

the *slli* or *shift-left-logical-immediate* instruction performs a logical left shift by *uimm6* bits of the value in the *rc* register.

```
r[rc] = r[rc] << simm6
```

2.1.13. addib.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**01100**	**00**

addib.i64 rc, rb, ib(uimm3*8)

the *addib* or *add-immediate-block* instruction loads a 64-bit constant addressed by *ib + uimm3 * 8* then adds it to the *rb* register and saves the result in the *rc* register.

```
r[rc] = r[rb] + (i64)[ib + uimm3 * 8]
```

2.1.14. load.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**01101**	**00**

load.i64 rc, (uimm3*8)(rb)

the *load* instruction adds *uimm3 * 8* to the *rb* to form an address, then loads a 64-bit value from memory at that address and saves the result in the *rc* register.

```
r[rc] = (i64)[r[rb] + uimm3 * 8]
```

2.1.15. loadib.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**01110**	**00**

loadib.i64 rc, ib(uimm3*8)(rb)

the *loadib* instruction loads a 64-bit constant addressed by *ib + uimm6 * 8* and adds to the *rb* to form an address, then loads a 64-bit value from memory at that address and saves the result in the *rc* register.

```
r[rc] = (i64)[r[rb] + (i64)[ib + uimm3 * 8]]
```

2.1.16. compare.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**01111**	**00**

cmp.i64 rc, rb, fun3

the *compare* instruction performs a comparison between the value in *rb* and *rc* then saves the result to the *flag* register. the *compare* opcode is also used to perform conditional move whereby the *rb* register is copied into the *rc* if the *flag* register is set. the type of comparsion in *fun3* can be one of: *0 →less than (signed)*, *1 →greather or equal (signed)*, *2 →equal*, *3 →not equal*, *4 →less than (unsigned)*, *5 →greater or equal (unsigned)*, or *6 →conditional move*.

```
match fun3
| lt  -> flag = (i64)r[rc] <  (i64)r[rb]
| ge  -> flag = (i64)r[rc] >= (i64)r[rb]
| eq  -> flag =      r[rc] =       r[rb]
| ne  -> flag =      r[rc] !=      r[rb]
| ltu -> flag = (u64)r[rc] <  (u64)r[rb]
| geu -> flag = (u64)r[rc] >= (u64)r[rb]
| mov -> if (flag) r[rc] = r[rb]
```

2.1.17. subib.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**10000**	**00**

subib.i64 rc, rb, ib(uimm3*8)

the *subib* or *sub-immediate-block* instruction loads a 64-bit constant addressed by *ib + uimm3 * 8* then subtracts it from the *rb* register and saves the result in the *rc* register.

```
r[rc] = r[rb] - (i64)[ib + uimm3 * 8]
```

2.1.18. store.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**10001**	**00**

store.i64 rc, (uimm3*8)(rb)

the *store* instruction adds *uimm3 * 8* to the *rb* register to form an address, then stores to memory at that address a 64-bit value from the *rc* register.

```
(i64)[r[rb] + uimm3 * 8] = r[rc]
```

2.1.19. storeib.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**10010**	**00**

storeib.i64 rc, ib(uimm3*8)(rb)

the *storeib* instruction loads a 64-bit constant addressed by *ib + uimm6 * 8* and adds to the the *rb* register to form an address, then stores to memory at that address a 64-bit value from the *rc* register.

```
(i64)[r[rb] + (i64)[ib + uimm3 * 8]] = r[rc]
```

2.1.20. logic.i64

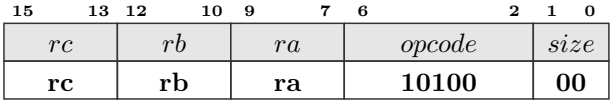
15 13	12 10	9 7	6 2	1 0
rc	*rb*	*imm3*	*opcode*	*size*
rc	**rb**	**uimm**	**10011**	**00**

logic.i64 rc, rb, fun3

the *logic* instruction performs a logic operation on the value in the *rb* register then stores the result in the *rc* register. the type of logic operations in *fun3* can be one of: *0 →move*, *1 →logical not*, *2 →negate*, *3 →bswap*, *4 →count trailing zeros*, *5 →count leading zeros*, or *6 →count population.*

```
match fun3
| mov   -> r[rc] = r[rb]
| not   -> r[rc] = ~r[rb]
| neg   -> r[rc] = -r[rb]
| bswap -> r[rc] = bswap(r[rb])
| ctz   -> r[rc] = ctz(r[rb])
| clz   -> r[rc] = clz(r[rb])
| ctpop -> r[rc] = ctpop(r[rb])
```

2.1.21. pin.i64



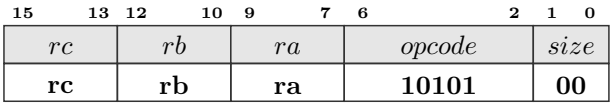
15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**10100**	**00**

pin.i64 rc, rb, ra

the *pin* or *pack-indirect* instruction packs two absolute addresses as an *i32x2 (pc,ib)* relative address vector. the register *ra* is subtracted from the *program counter + 2*, and the register *rb* is subtracted from *immediate base register*, and the results are packed into an *i32x2* relative address vector and saved to the register *rc*.

```
rpc = (i32)(pc - r[ra] + 2)
rib = (i32)(ib - r[rb]    )
r[rc] = (i32x2){rpc,rib}
```

2.1.22. and.i64



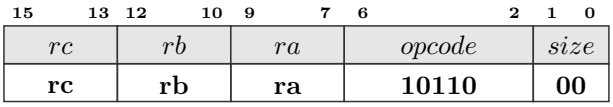
15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**10101**	**00**

and.i64 rc, rb, ra

the *and* instruction performs a *logical-and* of the register *rb* and the register *ra* and saves the result in the register *rc*.

```
r[rc] = r[rb] & r[ra]
```

2.1.23. or.i64



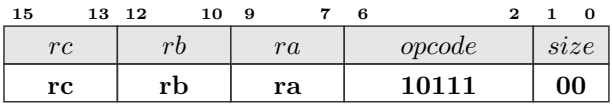
15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**10110**	**00**

or.i64 rc, rb, ra

the *or* instruction performs a *logical-or* of the register *rb* and the register *ra* and saves the result in the register *rc*.

```
r[rc] = r[rb] | r[ra]
```

2.1.24. xor.i64



15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**10111**	**00**

xor.i64 rc, rb, ra

the *xor* instruction performs a *logical-exclusive-or* of the register *rb* and the register *ra* and saves the result in the register *rc*.

```
r[rc] = r[rb] ^ r[ra]
```

2.1.25. sub.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**11000**	**00**

sub.i64 rc, rb, ra

the *sub* instruction subtracts the *ra* register from the *rb* register and saves the result in the *rc* register.

```
r[rc] = r[rb] - r[ra]
```

2.1.26. srl.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**11001**	**00**

srl.i64 rc, rb, ra

the *srl* or *shift-right-logical* instruction performs a logical right shift of the value in the *rb* register by the number of bits in register *ra* then saves the result in the *rc* register. zeros are copied into the right most bits.

```
r[rc] = (u64)r[rb] >> r[ra]
```

2.1.27. sra.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**11010**	**00**

sra.i64 rc, rb, ra

the *sra* or *shift-right-arithmetic* instruction performs an arithmetic right shift of the value in the *rb* register by the number of bits in register *ra* then saves the result in the *rc* register. sign is copied into the right most bits.

```
r[rc] = (i64)r[rb] >> r[ra]
```

2.1.28. sll.i64

15 13	12 10	9 7	6 2	1 0
rc	*rb*	*ra*	*opcode*	*size*
rc	**rb**	**ra**	**11011**	**00**

sll.i64 rc, rb, ra

the *sll* or *shift-left-logical* instruction performs a logical left shift of the value in the *rb* register by the number of bits in register *ra* then saves the result in the *rc* register.

```
r[rc] = r[rb] << r[ra]
```

2.1.29. add.i64

15 13	12 10	9 7	6 2	1 0	
rc	*rb*	*ra*	*opcode*	*size*	
rc	**rb**	**ra**	**11100**	**00**	**add.i64** rc, rb, ra

the *add* instruction adds the *rb* register to the *ra* register and saves the result in the *rc* register.

```
r[rc] = r[rb] + r[ra]
```

2.1.30. mul.i64

15 13	12 10	9 7	6 2	1 0	
rc	*rb*	*ra*	*opcode*	*size*	
rc	**rb**	**ra**	**11101**	**00**	**mul.i64** rc, rb, ra

the *mul* instruction performs signed multiplication of the *rb* register with the *ra* register and saves the result in the *rc* register.

```
r[rc] = r[rb] * r[ra]
```

2.1.31. div.i64

15 13	12 10	9 7	6 2	1 0	
rc	*rb*	*ra*	*opcode*	*size*	
rc	**rb**	**ra**	**11110**	**00**	**div.i64** rc, rb, ra

the *div* instruction performs signed division of the *rb* register by the *ra* register and saves the result in the *rc* register. division by zero causes a divide by zero exception.

```
r[rc] = r[rb] / r[ra]
```

2.1.32. illegal

15 7	6 2	1 0	
imm9	*opcode*	*size*	
uimm	**11111**	**00**	**illegal** uimm9

the *illegal* instruction causes an illegal instruction trap.

A. Appendix

A.1. 16-bit opcodes

15 13	12 10	9 7	6 2	1 0	
uimm			**00000**	**00**	**break** uimm9
simm			**00001**	**00**	**j** simm9
simm			**00010**	**00**	**b** simm9
simm			**00011**	**00**	**ibj** simm9
rc	**uimm**		**00100**	**00**	**jalib** rc, ib(uimm6*8)
rc	**uimm**		**00101**	**00**	**jtlib** rc, ib(uimm6*8)
rc	**uimm**		**00110**	**00**	**lib.i64** rc, ib(uimm6*8)
rc	**simm**		**00111**	**00**	**li.i64** rc, simm6
rc	**simm**		**01000**	**00**	**addi.i64** rc, simm6
rc	**uimm**		**01001**	**00**	**srli.i64** rc, uimm6
rc	**uimm**		**01010**	**00**	**srai.i64** rc, uimm6
rc	**uimm**		**01011**	**00**	**slli.i64** rc, uimm6
rc	**rb**	**uimm**	**01100**	**00**	**addib.i64** rc, rb, ib(uimm3*8)
rc	**rb**	**uimm**	**01101**	**00**	**load.i64** rc, (uimm3*8)(rb)
rc	**rb**	**uimm**	**01110**	**00**	**loadib.i64** rc, ib(uimm3*8)(rb)
rc	**rb**	**uimm**	**01111**	**00**	**compare.i64** rc, rb, fun3
rc	**rb**	**uimm**	**10000**	**00**	**subib.i64** rc, rb, ib(uimm3*8)
rc	**rb**	**uimm**	**10001**	**00**	**store.i64** rc, (uimm3*8)(rb)
rc	**rb**	**uimm**	**10010**	**00**	**storeib.i64** rc, ib(uimm3*8)(rb)
rc	**rb**	**uimm**	**10011**	**00**	**logic.i64** rc, rb, fun3
rc	**rb**	**ra**	**10100**	**00**	**pin.i64** rc, rb, ra
rc	**rb**	**ra**	**10101**	**00**	**and.i64** rc, rb, ra
rc	**rb**	**ra**	**10110**	**00**	**or.i64** rc, rb, ra
rc	**rb**	**ra**	**10111**	**00**	**xor.i64** rc, rb, ra
rc	**rb**	**ra**	**11000**	**00**	**sub.i64** rc, rb, ra
rc	**rb**	**ra**	**11001**	**00**	**srl.i64** rc, rb, ra
rc	**rb**	**ra**	**11010**	**00**	**sra.i64** rc, rb, ra
rc	**rb**	**ra**	**11011**	**00**	**sll.i64** rc, rb, ra
rc	**rb**	**ra**	**11100**	**00**	**add.i64** rc, rb, ra
rc	**rb**	**ra**	**11101**	**00**	**mul.i64** rc, rb, ra
rc	**rb**	**ra**	**11110**	**00**	**div.i64** rc, rb, ra
uimm			**11111**	**00**	**illegal** uimm9